

1. Téléchargez le contenu de la base de données (utilisez `tf.keras.datasets.cifar10`).

On s'assure de travailler dans un environnement qui contient les bibliothèques `tensorflow` et `keras`, et on importe `cifar10` à partir de la sous-bibliothèque `datasets` avec la ligne `from keras.datasets import cifar10`.

2. La base de données est répartie en des données d'entraînement et des données de test. Formez les deux sous-ensembles de données `x_train` et `x_test` correspondant respectivement aux données d'entraînement et de test.

Avec la fonction `cifar10.load_data()` on charge les 50000 images et étiquettes d'entraînements dans `x_train_images` et `x_train_labels`, et les 10000 images et étiquettes de test dans `x_test_images` et `x_test_labels`.

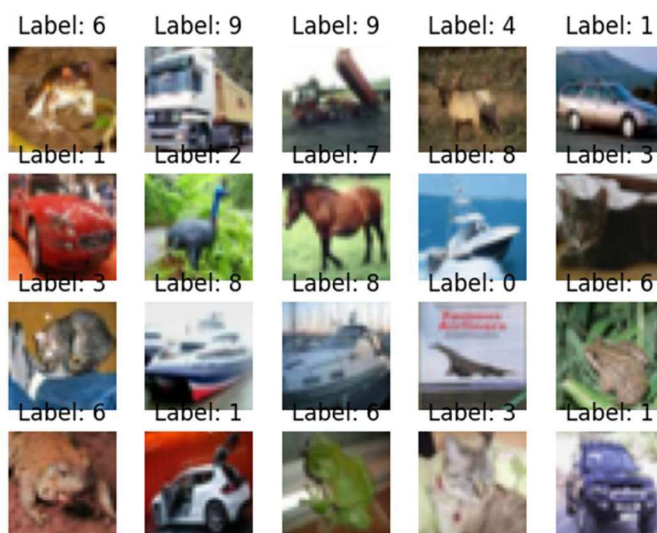
3. Vérifiez la dimension des données d'entraînement et de test, le nombre de classes et le nombre d'échantillons par classe.

On utilise `print()` pour afficher ces attributs dans la console pour notre bénéfice, mais on pourrait utiliser des `assert()` pour les vérifier efficacement sans produire de sortie. Le résultat dans la console est le suivant.

```
x_train_images shape: (50000, 32, 32, 3)
x_test_images shape: (10000, 32, 32, 3)
x_train_labels shape: (50000, 1)
x_test_labels shape: (10000, 1)
```

4. Affichez une dizaine d'échantillons de la base de données

On utilise `matplotlib.pyplot` pour afficher facilement les 10 premiers échantillons des deux sous-ensembles avec leurs étiquettes. On `import` cette bibliothèque, puis on utilise deux boucles pour itérer sur chaque sous-ensemble et afficher leurs 10 premières paires d'image-étiquette. Le résultat se retrouve dans cette fenêtre :



5. Réalisez une standardisation des deux sous-ensembles des données (normalisation des valeurs des pixels)

On standardise les deux ensembles de données en les divisant par 255 pour que chaque pixel ait une valeur entre 0 et 1.

6. Construisez un réseau de neurone convolutif (CNN) ayant l'architecture suivante :

- Utilisez trois filtres (couches de convolution) de dimension 3 qui progressent selon les couches selon : 32, puis 64, puis 128.
- Utilisez une normalisation par Batch après chaque couche de convolution et un maxPooling de taille 2
- Utilisez la fonction d'activation relu et un padding='same'.
- Utilisez ensuite le réseau intégralement connecté, constitué d'une couche dense de taille 1024
- Utilisez également un dropout avec un taux d'extinction de 20%

On commence par importer la bibliothèque du modèle `Sequential` et celles des couches demandées. On commence par implémenter les trois couches `Conv2D`, en précisant les tailles de filtres demandées, la taille de noyau de 3, la fonction d'activation `relu`, et le `padding`. Dans la première couche, on précise aussi la forme de l'entrée pour qu'elle soit la même que celle afficher avec le code de la question 3. Après chaque couche de convolution, on ajoute une normalisation par batch. Les deux premières convolutions sont ensuite suivies d'un `MaxPooling2D` de taille 2. La dernière est suivie d'un aplatissement afin de passer à une couche Dense de taille 1024. On ajoute un `Dropout` de 20% (soit 0.2). Finalement, on ajoute une couche finale Dense ayant la même taille que le nombre d'étiquettes.

7. Affichez le sommaire du modèle 1

On implémente `model.summary()` qui affiche le résultat suivant dans la console.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
conv2d_1 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 64)	256
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
conv2d_2 (Conv2D)	(None, 8, 8, 128)	73,856
batch_normalization_2 (BatchNormalization)	(None, 8, 8, 128)	512
flatten (Flatten)	(None, 8192)	0
dense (Dense)	(None, 1024)	8,389,632
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 10)	10,250

Total params: 8,494,026 (32.40 MB)
Trainable params: 8,493,578 (32.40 MB)
Non-trainable params: 448 (1.75 KB)

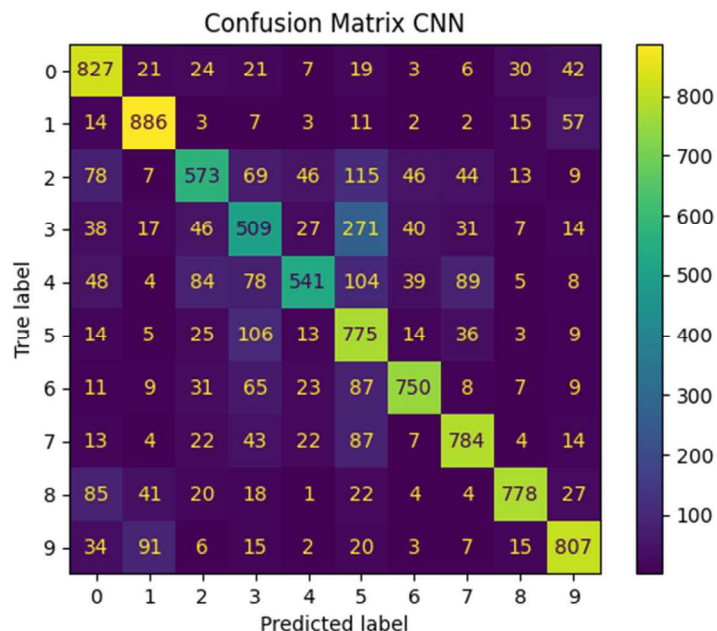
8. En utilisant l'optimisation adam, entraînez le CNN sur le jeu de données d'entraînement de CIFAR10 pendant 50 époques. La métrique utilisée étant la valeur de l'accuracy.

On compile donc le modèle avec l'optimisateur `adam` et la métrique `accuracy`.

Puisqu'on ne précise pas de fonction de perte, on utilise `sparse_categorical_crossentropy`. Ensuite, on entraîne le modèle avec les données de `x_train` pendant 50 époques. On utilise une taille de batch de 50 et on utilise les données de `x_test` pour valider l'entraînement à la fin de chaque époque. On met cet entraînement dans la variable `history` pour la question 10.

9. Représentez la matrice de confusion sur les données de test. Commentez les résultats.

On importe `numpy` et `scikit-learn` pour créer la matrice de confusion. On crée les prédictions d'étiquettes du modèle à partir des images de `x_test`. On utilise `argmax()` pour transformer les probabilités prédites en une liste des indices considérés les plus probables. On associe cette liste à la liste d'indice attendu dans `x_test_labels` afin de calculer la matrice de confusion.



On sait que le jeu de données d'entraînement contient 1000 images de chaque catégorie (pour un total de 10000 images). De ce fait, on peut remarquer que le modèle est particulièrement bon pour identifier correctement les images de catégories 1 (88.6% d'exactitude), 0 (82.7% d'exactitude) et 9 (80.7% d'exactitude); et il est particulièrement mauvais pour identifier correctement les images de catégories 3 (50.9% d'exactitude), 4 (54.1% d'exactitude) et 2 (57.3% d'exactitude).

Le modèle identifie incorrectement particulièrement souvent les images de catégories 2, 3, et 4 comme des images de catégorie 5 (11.5%, 27.1% et 10.4% du temps respectivement); et les images de catégorie 5 comme des images de

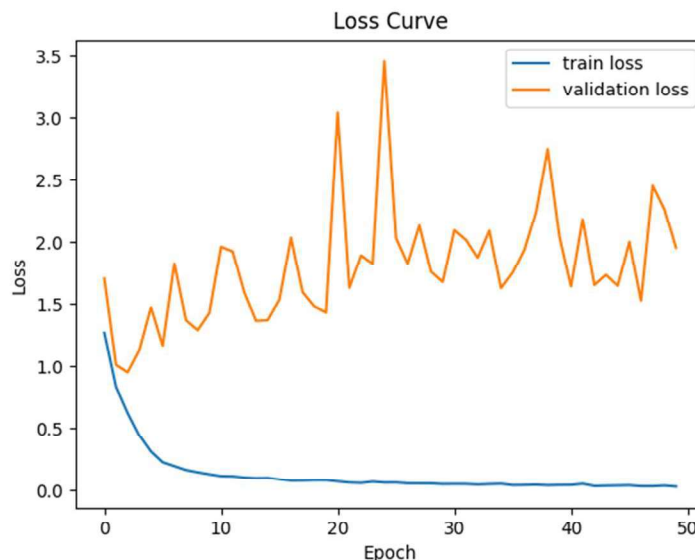
catégorie 3 (10.6% du temps). Toutes les autres erreurs de classification se produisent dans moins de 10% des cas.

Le modèle identifie incorrectement particulièrement rarement (moins de 1% du temps) les images des catégories 3,4,5,6 et 7 comme des images de catégorie 8; les images des catégories 0,1,7,8 et 9 comme des images de catégorie 6; les images des catégories 1 comme des images de catégorie 2,3,4,6 et 7; les images de catégories 0,1,8 et 9 pour des images de catégories 4 ou 7; les images de catégorie 6 comme des images de catégories 1,7,8 et 9; les images de catégories 9 comme des images de catégories 2,4,6 et 7; les images de catégories 4,5,6 et 7 comme des images de catégorie 1; et les images de catégorie 2,4,5 et 6 comme des images de catégorie 9.

On conclut donc que le modèle différencie très bien les images de catégories 1, 6 et 9 des autres dans les deux sens (extractions de caractéristiques uniques de la catégorie et identification d'image), et il différencie bien les catégories 4, 7 et 8 des autres images.

10. Représentez la courbe de variation de la fonction perte sur les données d'entraînement et de test.

On utilise `pyplot` qu'on a importé plus tôt. On utilise les données stockées lors que l'entraînement pour la variation de la perte à travers les époques pour les données d'entraînement et celles de validation. On obtient le graphique suivant.



On remarque que la perte de la validation augmente en moyenne à travers les époques alors que la perte de l'entraînement diminue et s'aplatit. Ceci indique un surapprentissage du modèle. Si l'apprentissage était bon, les deux courbes seraient similaires et diminueraient avant de se stabiliser.

11. Construisez un RNP dense constitué de 10 couches cachées, chacune avec 100 neurones. Utilisez l'initialisation de He et la fonction d'activation ELU. On crée un second modèle, composé uniquement de couches `Dense`. Pour les dix couches cachées, on précise une quantité d'unités de 100, une initialisation du

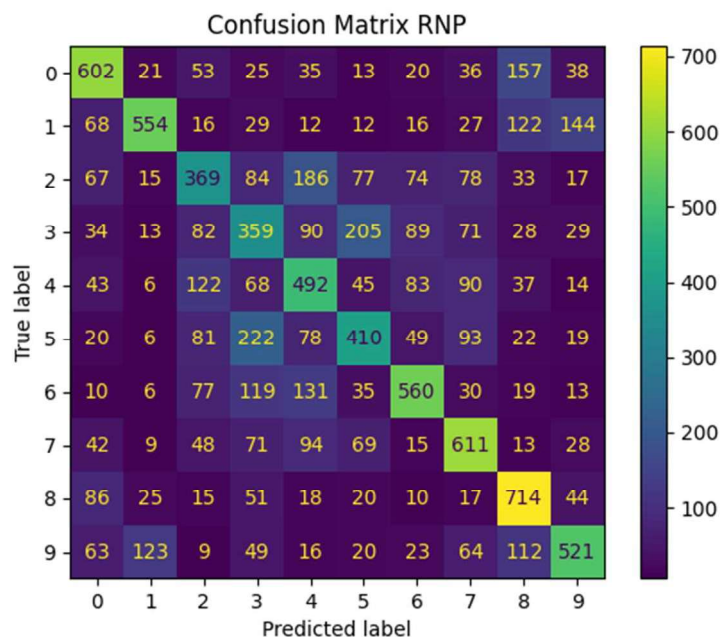
noyau de `he_normal` et la fonction d'activation `elu`. La couche de sortie est de 10 neurones afin d'être de la même taille que le nombre d'étiquettes.

12. En utilisant l'optimisation Nadam (`learning_rate=5e-5`), entraînez le RNP sur le jeu de données d'entraînement CIFAR10 pendant 50 époques. La métrique utilisée étant l'accuracy.

On compile le modèle avec optimiseur `Nadam` avec le taux d'apprentissage précisé. On utilise la même fonction de perte que pour le CNN, et on utilise la métrique précisée. Avant d'entraîner le modèle, on aplatit les images pour qu'elles puissent être manipulées par les couches `Dense` du modèle. Finalement on entraîne le modèle avec les données aplaties pendant 50 époques en utilisant des `batch` de 50, et les données de test pour la validation.

13. Représentez la matrice de confusion. Commentez les résultats.

On utilise le même code utilisé pour la question 9 afin de générer la matrice de confusion de ce modèle.



On sait toujours que le jeu de données d'entraînement contient 1000 images de chaque catégorie (pour un total de 10000 images). On peut donc affirmer que ce modèle est meilleur à identifier correctement les images de catégories 8 (71.4% d'exactitude), 7 (61.1% d'exactitude) et 0 (60.2% d'exactitude), et qu'il identifie correctement le moins souvent les images de catégories 3 (35.9% d'exactitude) et 3 (36.9% d'exactitude).

Le modèle identifie incorrectement le plus souvent les images de catégorie 5 comme des images de catégorie 3 (22.2% du temps), celles de catégorie 3 comme des images de catégorie 5 (20.5% du temps). De plus, entre 10% et 20% du temps, il identifie les images de 2 pour 4 (18.6%), de 0 pour 8 (15.7%), de 1 pour 9 (14.4%), de 6 pour 4 (13.1%), de 9 pour 1 (12.3%), de 4 pour 2 (12.2%), de 1 pour 8 (12.2%), de 6 pour 3 (11.9%) et de 9 pour 8 (11.2%). On remarque donc que le

modèle confond souvent les images de catégories 2 et 4 entre elles, et celles de catégories 1 et 9 entre elles.

Le modèle identifie incorrectement le plus rarement (moins de 2% du temps) les images de catégorie 1 pour des images de catégories 2,4,5 et 6; celles de catégorie 6 pour ces images de catégorie 0,1,8 et 9; celles de catégorie 8 pour ces images de catégorie 2,4,6 et 7; celles de catégories 2,3,4,5,6 et 7 pour des images de catégories 1; et celles de catégories 2,4,5 et 6 pour des images de catégories 9. On conclut donc que le modèle différencie bien les images et les caractéristiques de la catégorie 1 de la majorité des autres caractéristiques et images, les caractéristiques des catégories 6 et 8 des autres images, et les images de la catégorie 9 des autres catégories.

14. Comparez les deux modèles CNN et RNP obtenus et commentez les résultats. Si on se concentre sur les matrices de confusion, on remarque que le modèle RNP est beaucoup plus confus que le modèle CNN. En effet, les pires taux d'identification correcte du CNN se retrouvent en haut de la moyenne et la médiane des taux d'identification correcte du RNP. De plus, la matrice du CNN ne contient que 4 taux d'identification incorrects dépassant 10%, alors que la matrice RPN en contient 11. Donc, non seulement le CNN est-il correct plus souvent, il est aussi incorrect moins souvent que le RPN.
- De plus, la matrice de confusion du CNN montre qu'il est fréquent pour le CNN de se tromper sur moins de 10 images pour plusieurs catégories, alors que c'est rarement le cas pour le RPN, qui se trompe sur beaucoup plus d'images, peu importe les catégories.
- Le seul cas où les deux matrices identifient rarement incorrectement les images est avec la catégorie 1, qui dans les deux cas semble uniquement être parfois confuse avec la catégorie 9. Cependant, la tendance du RPN de faire plus d'erreurs se sent aussi ici, où il assigne erronément à 1 plus de 10% d'image de 9, et à 9 plus de 10% d'image de 2.
- Finalement, on remarque que presque toutes les catégories se démarquent dans la matrice du CNN, que ce soit par leur taux élevé d'identification correcte ou leur taux très bas d'identification incorrecte entre deux catégories différentes. Le même ne peut pas être dit de la matrice de RPN, où les identifications incorrectes sont plus nombreuses et diffuses, ce qui indique que le modèle ne prédit pas bien les catégories et ne connaît pas les caractéristiques qui les rendent uniques.
- Cependant, en ce qui concerne le temps et les ressources nécessaires à l'entraînement des modèles, le modèle RPN prend beaucoup moins de temps (environ 10s par époque) que le modèle CNN (environ 180s par époque) sur la même machine avec le même nombre de ressource disponible. Donc l'exactitude du modèle CNN a un prix mesurable.

Code source, fichier projet-final.py dans le dossier deeplearning dans l'environnement python contenant toutes les librairies nécessaires :
/deeplearning/projet-final.py

```
##Q1
# Importation des données de cifar10
from keras.datasets import cifar10

##Q2
# Chargement du dataset
(x_train_images, x_train_labels), (x_test_images,
x_test_labels) = cifar10.load_data()

##Q3
# Vérification du format des données
print("x_train_images shape: ", x_train_images.shape)
print("x_test_images shape: ", x_test_images.shape)
print("x_train_labels shape: ", x_train_labels.shape)
print("x_test_labels shape: ", x_test_labels.shape)
# Vérification alternative du format des données, moins
verbose
#assert x_train_images.shape == (50000, 32, 32, 3)
#assert x_test_images.shape == (10000, 32, 32, 3)
#assert x_train_labels.shape == (50000, 1)
#assert x_test_labels.shape == (10000, 1)

##Q4
# Affichage des 10 premiers échantillons de x_train et x_test
# pyplot pour visualiser les images
import matplotlib.pyplot as plt

for i in range(10):
    plt.subplot(4, 5, i + 1)
    plt.imshow(x_train_images[i])
    plt.title(f'Label: {x_train_labels[i][0]}')
    plt.axis('off')
for i in range(10):
    plt.subplot(4, 5, i + 11)
    plt.imshow(x_test_images[i])
    plt.title(f'Label: {x_test_labels[i][0]}')
    plt.axis('off')
plt.savefig('deeplearning/figures/first_10_samples.png')
plt.close()

##Q5
# Standardisation des données
x_train_images = x_train_images.astype('float32') / 255
x_test_images = x_test_images.astype('float32') / 255

##Q6
```

```
# Construction du CNN
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D,
BatchNormalization, MaxPooling2D, Dropout

cnn = Sequential()
cnn.add(Conv2D(filters=32, kernel_size=3, activation='relu',
padding='same', input_shape=(32, 32, 3)))
cnn.add(BatchNormalization())
cnn.add(MaxPooling2D(pool_size=2))
cnn.add(Conv2D(filters=64, kernel_size=3, activation='relu',
padding='same'))
cnn.add(BatchNormalization())
cnn.add(MaxPooling2D(pool_size=2))
cnn.add(Conv2D(filters=128, kernel_size=3, activation='relu',
padding='same'))
cnn.add(BatchNormalization())
cnn.add(Flatten())
cnn.add(Dense(1024, activation='relu'))
cnn.add(Dropout(0.2))
cnn.add(Dense(10, activation='softmax'))

##Q7
# Affichage du sommaire du modèle
cnn.summary()

##Q8
# Compilation et entraînement du modèle
cnn.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
history = cnn.fit(x_train_images, x_train_labels, epochs=50,
batch_size=50, validation_data=(x_test_images, x_test_labels),
verbose=1)

##Q9
# Matrice de confusion sur x_test
import numpy as np
from sklearn.metrics import confusion_matrix,
ConfusionMatrixDisplay

predictions_cnn = cnn.predict(x_test_images)
y_pred_cnn = np.argmax(predictions_cnn, axis=1)
cm_cnn = confusion_matrix(x_test_labels, y_pred_cnn)
disp_cnn = ConfusionMatrixDisplay(confusion_matrix=cm_cnn)
disp_cnn.plot()
plt.set_title("Confusion Matrix CNN")
plt.savefig('deeplearning/figures/confusion_matrix_cnn.png')
plt.close()

##Q10
```

```
# Courbe de la fonction de perte pou x_train et x_test
# on a déjà importé matplotlib.pyplot plus tôt donc on s'en
re-sert
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='validation loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Loss Curve')
plt.savefig('deeplearning/figures/loss_curve.png')
plt.close()
```

```
##Q11
# Construction du RNP
rnp = Sequential()
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu', input_shape=(32*32*3,)))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=100, kernel_initializer='he_normal',
activation='elu'))
rnp.add(Dense(units=10, activation='softmax'))
```

```
##Q12
# Compilation et entrainement du modèle
from keras import optimizers

# compilation du modèle
rnp.compile(optimizer=optimizers.Nadam(learning_rate=0.00005),
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
# reformattage les données pour le RNP
x_train_flat = x_train_images.reshape(x_train_images.shape[0],
-1)
x_test_flat = x_test_images.reshape(x_test_images.shape[0], -
1)
# entrainement du modèle
```

```
rnp.fit(x_train_flat, x_train_labels, epochs=50,  
batch_size=50, validation_data=(x_test_flat, x_test_labels),  
verbose=1)
```

```
##Q13  
# Matrice de confusion sur x_test  
# on a déjà importe scikit-learn donc on s'en re-sert  
predictions_rnp = rnp.predict(x_test_flat)  
y_pred_rnp = np.argmax(predictions_rnp, axis=1)  
cm_rnp = confusion_matrix(x_test_labels, y_pred_rnp)  
disp_rnp = ConfusionMatrixDisplay(confusion_matrix=cm_rnp)  
disp_rnp.plot()  
plt.set_title("Confusion Matrix RNP")  
plt.savefig('deeplearning/figures/confusion_matrix_rnp.png')  
plt.close()
```

Résultat dans la console de la commande &

```
c:/Users/loubo/Documents/UNI/TELUQ/E26/SCI1035/deeplearning/Sc  
ripts/python.exe  
C:\Users\loubo\Documents\UNI\TELUQ\E26\SCI1035\deeplearning\pr  
ojet-final.py
```

```
WARNING: All log messages before absl::InitializeLog() is  
called are written to STDERR  
I0000 00:00:1786055201.540110    13336 port.cc:153] oneDNN  
custom operations are on. You may see slightly different  
numerical results due to floating-point round-off errors from  
different computation orders. To turn them off, set the  
environment variable `TF_ENABLE_ONEDNN_OPTS=0`.  
WARNING: All log messages before absl::InitializeLog() is  
called are written to STDERR  
I0000 00:00:1786055204.459090    13336 port.cc:153] oneDNN  
custom operations are on. You may see slightly different  
numerical results due to floating-point round-off errors from  
different computation orders. To turn them off, set the  
environment variable `TF_ENABLE_ONEDNN_OPTS=0`.  
x_train_images shape: (50000, 32, 32, 3)  
x_test_images shape: (10000, 32, 32, 3)  
x_train_labels shape: (50000, 1)  
x_test_labels shape: (10000, 1)  
C:\Users\loubo\Documents\UNI\TELUQ\E26\SCI1035\deeplearning\li  
b\site-  
packages\keras\src\layers\convolutional\base_conv.py:113:  
UserWarning: Do not pass an `input_shape`/`input_dim` argument  
to a layer. When using Sequential models, prefer using an  
`Input(shape)` object as the first layer in the model instead.  
    super().__init__(activity_regularizer=activity_regularizer,  
**kwargs)  
I0000 00:00:1786055208.969514    13336  
cpu_feature_guard.cc:227] This TensorFlow binary is optimized
```

to use available CPU instructions in performance-critical operations.

To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

Model: "sequential"

Layer (type)	Output Shape
Param #	
conv2d (Conv2D)	(None, 32, 32, 32)
896	
batch_normalization	(None, 32, 32, 32)
128	
(BatchNormalization)	
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)
0	
conv2d_1 (Conv2D)	(None, 16, 16, 64)
18,496	
batch_normalization_1	(None, 16, 16, 64)
256	
(BatchNormalization)	
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)
0	
conv2d_2 (Conv2D)	(None, 8, 8, 128)
73,856	
batch_normalization_2	(None, 8, 8, 128)
512	

(BatchNormalization)	
flatten (Flatten)	(None, 8192)
0	
dense (Dense)	(None, 1024)
8,389,632	
dropout (Dropout)	(None, 1024)
0	
dense_1 (Dense)	(None, 10)
10,250	

Total params: 8,494,026 (32.40 MB)

Trainable params: 8,493,578 (32.40 MB)

Non-trainable params: 448 (1.75 KB)

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.

Epoch 1/50

1000/1000 176s 174ms/step -

accuracy: 0.5685 - loss: 1.2671 - val_accuracy: 0.4776 -

val_loss: 1.7044

Epoch 2/50

1000/1000 175s 175ms/step -

accuracy: 0.7116 - loss: 0.8338 - val_accuracy: 0.6583 -

val_loss: 1.0104

Epoch 3/50

1000/1000 170s 170ms/step -

accuracy: 0.7848 - loss: 0.6214 - val_accuracy: 0.6985 -

val_loss: 0.9513

Epoch 4/50

1000/1000 179s 179ms/step -

accuracy: 0.8478 - loss: 0.4390 - val_accuracy: 0.6711 -

val_loss: 1.1297

Epoch 5/50

1000/1000 183s 183ms/step -

accuracy: 0.8915 - loss: 0.3102 - val_accuracy: 0.6117 -

val_loss: 1.4719

Epoch 6/50

```
1000/1000 ----- 190s 190ms/step -  
accuracy: 0.9241 - loss: 0.2251 - val_accuracy: 0.7183 -  
val_loss: 1.1640  
Epoch 7/50  
1000/1000 ----- 170s 170ms/step -  
accuracy: 0.9343 - loss: 0.1937 - val_accuracy: 0.6590 -  
val_loss: 1.8281  
Epoch 8/50  
1000/1000 ----- 165s 165ms/step -  
accuracy: 0.9455 - loss: 0.1628 - val_accuracy: 0.7259 -  
val_loss: 1.3685  
Epoch 9/50  
1000/1000 ----- 200s 200ms/step -  
accuracy: 0.9533 - loss: 0.1446 - val_accuracy: 0.7356 -  
val_loss: 1.2889  
Epoch 10/50  
1000/1000 ----- 211s 212ms/step -  
accuracy: 0.9597 - loss: 0.1288 - val_accuracy: 0.7347 -  
val_loss: 1.4271  
Epoch 11/50  
1000/1000 ----- 198s 198ms/step -  
accuracy: 0.9634 - loss: 0.1141 - val_accuracy: 0.6907 -  
val_loss: 1.9620  
Epoch 12/50  
1000/1000 ----- 188s 188ms/step -  
accuracy: 0.9644 - loss: 0.1123 - val_accuracy: 0.7076 -  
val_loss: 1.9254  
Epoch 13/50  
1000/1000 ----- 185s 185ms/step -  
accuracy: 0.9681 - loss: 0.1028 - val_accuracy: 0.7207 -  
val_loss: 1.5936  
Epoch 14/50  
1000/1000 ----- 178s 178ms/step -  
accuracy: 0.9691 - loss: 0.0979 - val_accuracy: 0.7304 -  
val_loss: 1.3637  
Epoch 15/50  
1000/1000 ----- 176s 176ms/step -  
accuracy: 0.9701 - loss: 0.0996 - val_accuracy: 0.7139 -  
val_loss: 1.3690  
Epoch 16/50  
1000/1000 ----- 171s 171ms/step -  
accuracy: 0.9735 - loss: 0.0863 - val_accuracy: 0.7317 -  
val_loss: 1.5346  
Epoch 17/50  
1000/1000 ----- 152s 152ms/step -  
accuracy: 0.9766 - loss: 0.0756 - val_accuracy: 0.7262 -  
val_loss: 2.0344  
Epoch 18/50
```

```
1000/1000 ----- 179s 179ms/step -  
accuracy: 0.9772 - loss: 0.0764 - val_accuracy: 0.7332 -  
val_loss: 1.5942  
Epoch 19/50  
1000/1000 ----- 159s 159ms/step -  
accuracy: 0.9754 - loss: 0.0791 - val_accuracy: 0.7049 -  
val_loss: 1.4807  
Epoch 20/50  
1000/1000 ----- 164s 164ms/step -  
accuracy: 0.9759 - loss: 0.0791 - val_accuracy: 0.7364 -  
val_loss: 1.4306  
Epoch 21/50  
1000/1000 ----- 154s 154ms/step -  
accuracy: 0.9797 - loss: 0.0695 - val_accuracy: 0.6970 -  
val_loss: 3.0404  
Epoch 22/50  
1000/1000 ----- 185s 185ms/step -  
accuracy: 0.9817 - loss: 0.0603 - val_accuracy: 0.7436 -  
val_loss: 1.6307  
Epoch 23/50  
1000/1000 ----- 178s 178ms/step -  
accuracy: 0.9830 - loss: 0.0570 - val_accuracy: 0.7460 -  
val_loss: 1.8937  
Epoch 24/50  
1000/1000 ----- 169s 169ms/step -  
accuracy: 0.9802 - loss: 0.0674 - val_accuracy: 0.7224 -  
val_loss: 1.8276  
Epoch 25/50  
1000/1000 ----- 159s 159ms/step -  
accuracy: 0.9821 - loss: 0.0602 - val_accuracy: 0.7302 -  
val_loss: 3.4582  
Epoch 26/50  
1000/1000 ----- 160s 160ms/step -  
accuracy: 0.9818 - loss: 0.0607 - val_accuracy: 0.7442 -  
val_loss: 2.0325  
Epoch 27/50  
1000/1000 ----- 160s 160ms/step -  
accuracy: 0.9842 - loss: 0.0538 - val_accuracy: 0.7323 -  
val_loss: 1.8287  
Epoch 28/50  
1000/1000 ----- 160s 160ms/step -  
accuracy: 0.9846 - loss: 0.0536 - val_accuracy: 0.7452 -  
val_loss: 2.1333  
Epoch 29/50  
1000/1000 ----- 174s 174ms/step -  
accuracy: 0.9851 - loss: 0.0533 - val_accuracy: 0.7225 -  
val_loss: 1.7613  
Epoch 30/50
```

```
1000/1000 ----- 189s 189ms/step -  
accuracy: 0.9862 - loss: 0.0483 - val_accuracy: 0.7329 -  
val_loss: 1.6776  
Epoch 31/50  
1000/1000 ----- 186s 186ms/step -  
accuracy: 0.9860 - loss: 0.0491 - val_accuracy: 0.7460 -  
val_loss: 2.0947  
Epoch 32/50  
1000/1000 ----- 187s 187ms/step -  
accuracy: 0.9855 - loss: 0.0487 - val_accuracy: 0.7447 -  
val_loss: 2.0168  
Epoch 33/50  
1000/1000 ----- 185s 185ms/step -  
accuracy: 0.9873 - loss: 0.0441 - val_accuracy: 0.7491 -  
val_loss: 1.8765  
Epoch 34/50  
1000/1000 ----- 191s 191ms/step -  
accuracy: 0.9865 - loss: 0.0476 - val_accuracy: 0.7320 -  
val_loss: 2.0910  
Epoch 35/50  
1000/1000 ----- 185s 185ms/step -  
accuracy: 0.9862 - loss: 0.0504 - val_accuracy: 0.7425 -  
val_loss: 1.6285  
Epoch 36/50  
1000/1000 ----- 183s 183ms/step -  
accuracy: 0.9885 - loss: 0.0392 - val_accuracy: 0.7312 -  
val_loss: 1.7545  
Epoch 37/50  
1000/1000 ----- 184s 184ms/step -  
accuracy: 0.9882 - loss: 0.0405 - val_accuracy: 0.7263 -  
val_loss: 1.9379  
Epoch 38/50  
1000/1000 ----- 184s 184ms/step -  
accuracy: 0.9885 - loss: 0.0426 - val_accuracy: 0.7489 -  
val_loss: 2.2384  
Epoch 39/50  
1000/1000 ----- 184s 184ms/step -  
accuracy: 0.9885 - loss: 0.0385 - val_accuracy: 0.7457 -  
val_loss: 2.7445  
Epoch 40/50  
1000/1000 ----- 190s 190ms/step -  
accuracy: 0.9888 - loss: 0.0405 - val_accuracy: 0.7424 -  
val_loss: 2.0488  
Epoch 41/50  
1000/1000 ----- 168s 168ms/step -  
accuracy: 0.9883 - loss: 0.0409 - val_accuracy: 0.7191 -  
val_loss: 1.6413  
Epoch 42/50
```

```
1000/1000 ----- 160s 160ms/step -
accuracy: 0.9861 - loss: 0.0502 - val_accuracy: 0.7384 -
val_loss: 2.1837
Epoch 43/50
1000/1000 ----- 163s 163ms/step -
accuracy: 0.9901 - loss: 0.0326 - val_accuracy: 0.7482 -
val_loss: 1.6531
Epoch 44/50
1000/1000 ----- 176s 176ms/step -
accuracy: 0.9902 - loss: 0.0348 - val_accuracy: 0.7397 -
val_loss: 1.7353
Epoch 45/50
1000/1000 ----- 191s 190ms/step -
accuracy: 0.9896 - loss: 0.0364 - val_accuracy: 0.7300 -
val_loss: 1.6473
Epoch 46/50
1000/1000 ----- 186s 186ms/step -
accuracy: 0.9894 - loss: 0.0382 - val_accuracy: 0.7164 -
val_loss: 2.0020
Epoch 47/50
1000/1000 ----- 200s 200ms/step -
accuracy: 0.9907 - loss: 0.0310 - val_accuracy: 0.7278 -
val_loss: 1.5269
Epoch 48/50
1000/1000 ----- 191s 191ms/step -
accuracy: 0.9913 - loss: 0.0308 - val_accuracy: 0.7386 -
val_loss: 2.4519
Epoch 49/50
1000/1000 ----- 191s 191ms/step -
accuracy: 0.9900 - loss: 0.0358 - val_accuracy: 0.7511 -
val_loss: 2.2650
Epoch 50/50
1000/1000 ----- 192s 192ms/step -
accuracy: 0.9919 - loss: 0.0281 - val_accuracy: 0.7230 -
val_loss: 1.9561
313/313 ----- 4s 14ms/step
C:\Users\loubo\Documents\UNI\TELUQ\E26\SCI1035\deeplearning\li
b\site-packages\keras\src\layers\core\dense.py:95:
UserWarning: Do not pass an `input_shape`/`input_dim` argument
to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
Epoch 1/50
1000/1000 ----- 13s 11ms/step -
accuracy: 0.3173 - loss: 1.9002 - val_accuracy: 0.3859 -
val_loss: 1.7261
Epoch 2/50
```

```
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.4016 - loss: 1.6770 - val_accuracy: 0.4111 -  
val_loss: 1.6478  
Epoch 3/50  
1000/1000 ----- 10s 9ms/step -  
accuracy: 0.4320 - loss: 1.5926 - val_accuracy: 0.4350 -  
val_loss: 1.5853  
Epoch 4/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.4519 - loss: 1.5383 - val_accuracy: 0.4587 -  
val_loss: 1.5299  
Epoch 5/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.4676 - loss: 1.4950 - val_accuracy: 0.4555 -  
val_loss: 1.5242  
Epoch 6/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.4793 - loss: 1.4608 - val_accuracy: 0.4692 -  
val_loss: 1.4928  
Epoch 7/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.4908 - loss: 1.4323 - val_accuracy: 0.4710 -  
val_loss: 1.4927  
Epoch 8/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5009 - loss: 1.4054 - val_accuracy: 0.4715 -  
val_loss: 1.4918  
Epoch 9/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5073 - loss: 1.3830 - val_accuracy: 0.4844 -  
val_loss: 1.4521  
Epoch 10/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5165 - loss: 1.3601 - val_accuracy: 0.4920 -  
val_loss: 1.4413  
Epoch 11/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5202 - loss: 1.3423 - val_accuracy: 0.4946 -  
val_loss: 1.4294  
Epoch 12/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5282 - loss: 1.3251 - val_accuracy: 0.5016 -  
val_loss: 1.4178  
Epoch 13/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5346 - loss: 1.3065 - val_accuracy: 0.5008 -  
val_loss: 1.4101  
Epoch 14/50
```

```
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5404 - loss: 1.2904 - val_accuracy: 0.5057 -  
val_loss: 1.4042  
Epoch 15/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5455 - loss: 1.2756 - val_accuracy: 0.4929 -  
val_loss: 1.4434  
Epoch 16/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5516 - loss: 1.2611 - val_accuracy: 0.5061 -  
val_loss: 1.3987  
Epoch 17/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5543 - loss: 1.2475 - val_accuracy: 0.5116 -  
val_loss: 1.3989  
Epoch 18/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5598 - loss: 1.2352 - val_accuracy: 0.5098 -  
val_loss: 1.3943  
Epoch 19/50  
1000/1000 ----- 12s 12ms/step -  
accuracy: 0.5667 - loss: 1.2215 - val_accuracy: 0.5036 -  
val_loss: 1.4017  
Epoch 20/50  
1000/1000 ----- 12s 11ms/step -  
accuracy: 0.5707 - loss: 1.2090 - val_accuracy: 0.5124 -  
val_loss: 1.3861  
Epoch 21/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5767 - loss: 1.1963 - val_accuracy: 0.4991 -  
val_loss: 1.4302  
Epoch 22/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.5805 - loss: 1.1844 - val_accuracy: 0.5124 -  
val_loss: 1.3989  
Epoch 23/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5823 - loss: 1.1731 - val_accuracy: 0.5145 -  
val_loss: 1.3999  
Epoch 24/50  
1000/1000 ----- 11s 11ms/step -  
accuracy: 0.5885 - loss: 1.1622 - val_accuracy: 0.5099 -  
val_loss: 1.3982  
Epoch 25/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5925 - loss: 1.1512 - val_accuracy: 0.5200 -  
val_loss: 1.3760  
Epoch 26/50
```

```
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5964 - loss: 1.1415 - val_accuracy: 0.5109 -  
val_loss: 1.4090  
Epoch 27/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.5968 - loss: 1.1315 - val_accuracy: 0.5181 -  
val_loss: 1.3870  
Epoch 28/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6021 - loss: 1.1226 - val_accuracy: 0.5192 -  
val_loss: 1.3830  
Epoch 29/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6073 - loss: 1.1101 - val_accuracy: 0.5169 -  
val_loss: 1.3925  
Epoch 30/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6095 - loss: 1.1014 - val_accuracy: 0.5266 -  
val_loss: 1.3771  
Epoch 31/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6133 - loss: 1.0929 - val_accuracy: 0.5172 -  
val_loss: 1.3823  
Epoch 32/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6162 - loss: 1.0807 - val_accuracy: 0.5119 -  
val_loss: 1.3991  
Epoch 33/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6197 - loss: 1.0737 - val_accuracy: 0.5236 -  
val_loss: 1.3910  
Epoch 34/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6240 - loss: 1.0655 - val_accuracy: 0.5129 -  
val_loss: 1.4004  
Epoch 35/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6266 - loss: 1.0523 - val_accuracy: 0.5258 -  
val_loss: 1.3850  
Epoch 36/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6305 - loss: 1.0466 - val_accuracy: 0.5182 -  
val_loss: 1.4196  
Epoch 37/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6335 - loss: 1.0346 - val_accuracy: 0.5155 -  
val_loss: 1.4124  
Epoch 38/50
```

```
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6369 - loss: 1.0283 - val_accuracy: 0.5204 -  
val_loss: 1.4109  
Epoch 39/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6390 - loss: 1.0225 - val_accuracy: 0.5215 -  
val_loss: 1.3964  
Epoch 40/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6426 - loss: 1.0112 - val_accuracy: 0.5193 -  
val_loss: 1.4062  
Epoch 41/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6464 - loss: 1.0030 - val_accuracy: 0.5167 -  
val_loss: 1.4237  
Epoch 42/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6500 - loss: 0.9953 - val_accuracy: 0.5223 -  
val_loss: 1.4221  
Epoch 43/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6520 - loss: 0.9875 - val_accuracy: 0.5179 -  
val_loss: 1.4235  
Epoch 44/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6564 - loss: 0.9796 - val_accuracy: 0.5168 -  
val_loss: 1.4253  
Epoch 45/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6598 - loss: 0.9696 - val_accuracy: 0.5229 -  
val_loss: 1.4221  
Epoch 46/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6614 - loss: 0.9617 - val_accuracy: 0.5183 -  
val_loss: 1.4329  
Epoch 47/50  
1000/1000 ----- 10s 10ms/step -  
accuracy: 0.6636 - loss: 0.9548 - val_accuracy: 0.5159 -  
val_loss: 1.4421  
Epoch 48/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6676 - loss: 0.9481 - val_accuracy: 0.5192 -  
val_loss: 1.4363  
Epoch 49/50  
1000/1000 ----- 9s 9ms/step -  
accuracy: 0.6700 - loss: 0.9400 - val_accuracy: 0.5190 -  
val_loss: 1.4510  
Epoch 50/50
```

```
1000/1000 _____ 10s 10ms/step -  
accuracy: 0.6737 - loss: 0.9295 - val_accuracy: 0.5192 -  
val_loss: 1.4408  
313/313 _____ 1s 4ms/step
```